

Full Stack Development with MERN

Project Documentation

1. Introduction

Project Title: Customer Registry System

Member : Guvva Swathi

2. Project Overview

Purpose

The Customer Registry System is a web-based application developed to manage customer complaints efficiently. It enables customers to register complaints, track their status, and receive updates, while administrators can manage and resolve complaints through a centralized platform.

Features

- User Registration and Login
- Complaint Submission
- Complaint Status Tracking
- Admin Dashboard
- Complaint Management
- User Management
- Secure Authentication
- Responsive User Interface

3. Architecture

Frontend

The frontend is developed using React.js. It provides an interactive and responsive interface for customers and administrators to access system functionalities.

Backend

The backend is built using Node.js and Express.js. It handles API requests, user authentication, complaint processing, and database communication.

Database

MongoDB is used as the database to store user information, complaints, complaint status updates, and administrative records.

4. Setup Instructions

Prerequisites

- Node.js
- MongoDB
- npm (Node Package Manager)
- VS Code

Installation

1. Clone the project repository.
2. Install dependencies using npm install.
3. Configure environment variables.
4. Start MongoDB service.
5. Run frontend and backend servers.

5. Folder Structure

Client

Contains React components, pages, services, styles, and routing configuration.

Server

Contains Express routes, controllers, models, middleware, database configuration, and API logic.

6. Running the Application

Frontend

cd client

npm install

npm start

Backend

cd server

npm install

npm start

7. API Documentation

Authentication APIs

- POST /api/auth/register
- POST /api/auth/login

Complaint APIs

- POST /api/complaints
- GET /api/complaints
- GET /api/complaints/:id
- PUT /api/complaints/:id

User APIs

- GET /api/users
- PUT /api/users/:id
- DELETE /api/users/:id

8. Authentication

Authentication is implemented using JWT (JSON Web Tokens). Users must log in with valid credentials to access protected resources. Passwords are securely stored using hashing techniques.

9. User Interface

The application provides:

- Login Page
- Registration Page
- Customer Dashboard
- Complaint Submission Form
- Complaint Tracking Page
- Admin Dashboard
- Complaint Management Page

10. Testing

The application was tested using:

- Functional Testing
- User Acceptance Testing
- API Testing using Postman
- Form Validation Testing

11. Screenshots or Demo

Include screenshots of:

- Login Page
- Registration Page
- Customer Dashboard
- Complaint Form
- Complaint Tracking
- Admin Dashboard

12. Known Issues

- Internet connection is required for application access.
- Large numbers of complaints may affect performance on low-end systems.

13. Future Enhancements

- Email Notifications
- SMS Alerts
- Complaint Analytics Dashboard
- AI-based Complaint Categorization
- Mobile Application Support
- Multi-language Support